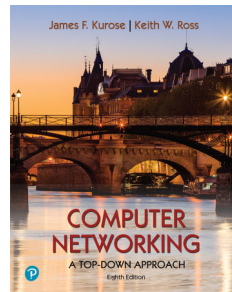


第2章 应用层

中国科学技术大学
自动化系 郑烱
改编自Jim kurose, Keith Ross

2025中科大网络



Computer Networking: A Top-Down Approach
8th edition
Jim Kurose, Keith Ross
Pearson, 2020

Application Layer: 2-1

应用层

2025中科大网络

目标:

- 应用层协议：概念和实现
 - 传输层的服务模型
 - 客户端/服务器模式
 - 对等体模式
- 应用层协议实例：通过研究支撑流行应用的协议和架构来学习
 - HTTP
 - SMTP, IMAP
 - DNS
 - 视频流化服务系统, CDNs
- 网络应用编程
 - Socket API

Application Layer: 2-2

应用层

2025中科大网络

- 网络应用原理
- Web和HTTP
- E-mail, SMTP, IMAP
- DNS (The Domain Name System)
- P2P 应用
- 视频流化和CDN (content distribution networks)
- UDP和TCP Socket 编程

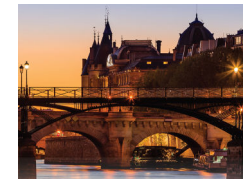


Application Layer: 2-3

应用层

2025中科大网络

- 网络应用原理
- Web和HTTP
- E-mail, SMTP, IMAP
- DNS (The Domain Name System)
- P2P 应用
- 视频流化和CDN (Content Distribution Networks)
- UDP和TCP Socket 编程



Application Layer: 2-4

一些网络应用

2025中科大网络

- 社交媒体
- Web
- 短报文系统
- e-mail
- 多用户网络游戏
- 流式存储视频服务 (YouTube, Hulu, Netflix)
- P2P 文件分发
- Voice over IP (e.g., Skype)
- 实时视频会议 (e.g., Zoom, 腾讯, 钉钉)
- 互联网搜索
- 远程登录
- ...

Q: 你最喜欢的是?

Application Layer: 2-5

创建新的网络应用

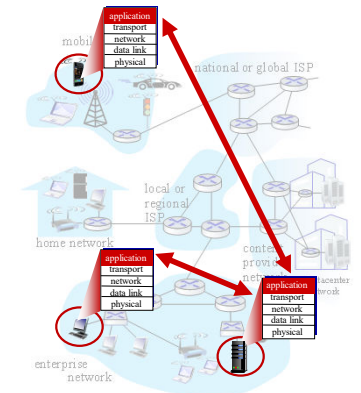
2025中科大网络

编写程序:

- 在 (不同的) 端系统上运行
- 通过网络进行通信
- e.g., web浏览器和服务器软件

在网络核心中无需部署所编写的软件

- 网络核心设备中不运行用户的应用软件
- 软件运行在端系统中, 允许快速的应用部署和传播



Application Layer: 2-6

客户端/服务器模式

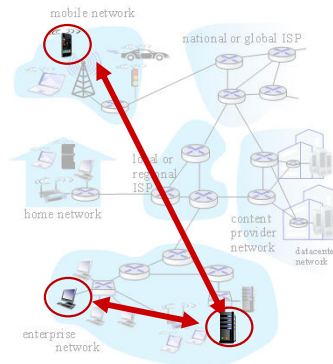
2025中科大网络

服务器:

- 在主机系统中始终运行
- 固定IP地址
- 部署在数据中心, 方便扩展

客户端:

- 与服务器进行联络和通信
- 可能与互联网有间歇性的连接
- 可能是动态IP地址
- 客户端之间通常不直接通信
- 例子: HTTP, IMAP, FTP

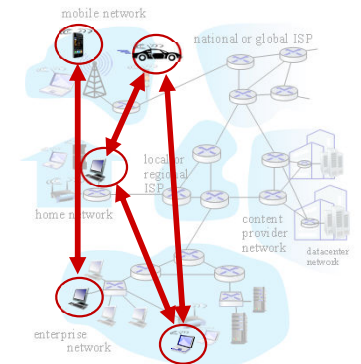


Application Layer: 2-7

对等体(P2P)架构

2025中科大网络

- (几乎) 没有一直运行的服务器
- 任意端系统之间可以进行通信
- peer请求其它peer的服务, 同时也向其它peer提供服务
 - 自扩展性: 新peer节点带来带来新的服务请求, 同时也带来新的服务能力
- Peers具有间歇性连接且可能有变化的IP地址
 - 管理复杂
- 例子: P2P 文件共享



Application Layer: 2-8

进程间通信

2025中科大网络

进程: 运行在端系统中的程序

- 在同一个主机内，2个应用进程使用**进程间通信机制**通信（由操作系统规范）

远程主机进程间通信：通过网络交换**报文(Message)**来实现

- 通过应用层与传输层的接口
- 使用OS提供的远程进程间通信服务
- 按照应用协议交换报文，实现应用协议，实现应用

Clients, Servers

客户端进程: 发起通信的进程

服务器进程: 等待连接的进程

注意：P2P架构的应用也有客户端进程和服务端进程之分

Application Layer: 2-9

引入port进行进程编址

2025中科大网络

- 主机设备拥有32-bit IP 地址
- 为了接收报文，进程必须拥有一个**标识器identifier**
- Q**: 仅仅有了主机的IP地址，是不是可以足以标识进程？
 - 当然不行，一个主机上有很多进程在运行
- 标识器**包括与端系统上进程相关的**IP地址**和**端口号**
- 端口的例子：
 - HTTP server: 80
 - mail server: 25
- 为了发送HTTP报文给 gaia.cs.umass.edu web server:
 - IP address**: 128.119.245.12
 - port number**: 80

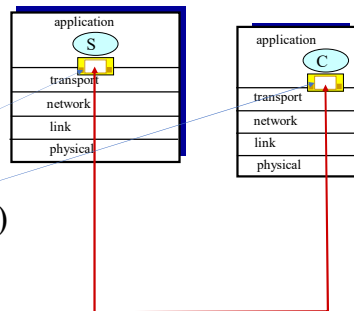
大楼号 类比 IP地址
房间号 类比 Port

Application Layer: 2-10

进程间的通信由2个端节点标识

2025中科大网络

- 不管TCP还是UDP上的进程间通信
- 2个进程间通信的由2个端节点(End Point)标识
- $= (IP + Port) + (IP' + Port')$

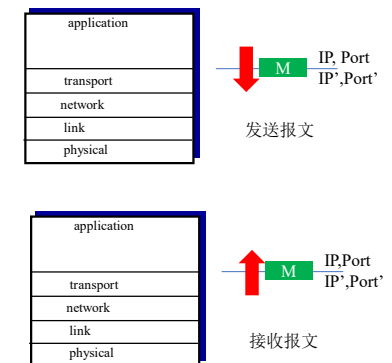


Socket: 11

传输层服务接口：携带的信息

2025中科大网络

- 应用报文发送和接收所需要携带的信息
 - 要传输的报文M: 货物
 - 本地IP, 本地Port: 发/收件人的所在地址 (houses)
 - 远程IP' 和远程Port': 收/发件人

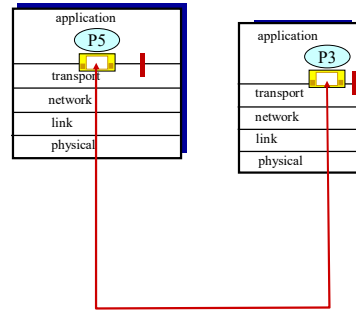


Socket: 12

穿过层间携带原始编址信息的问题

2025中科大网络

- 每次发送/接收穿过层间信息多
- 容易出错，不便于管理
 - 如果程序员提供端口前后不一致
 - 不便于OS维护状态:有没有连接建立?
- 用**整数**尽可能多地代表编址信息
 - 每次传M的时候，与此同时穿过层间接口还有用整数代表的编址信息
 - 信息量小，容易管理，不容易出错
 - 像文件句柄，端系统内约定：应用层和传输层关于进程编址的约定
 - 本地意义

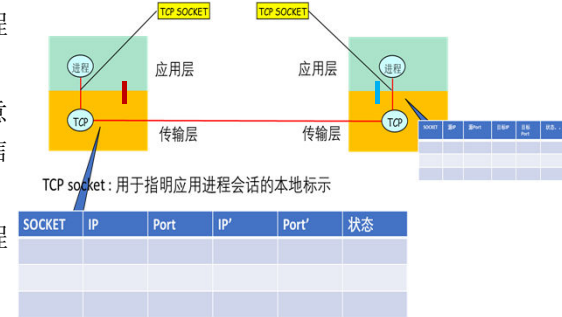


Socket: 13

TCP Socket=(IP,Port,IP',Port')

2025中科大网络

- TCP是面向连接的，进程间通信要持续一段时间
- 通信前有握手过程：同意和对方通信，准备好通信资源，值得
- 在交换报文时，两个进程间的通信关系已经确定

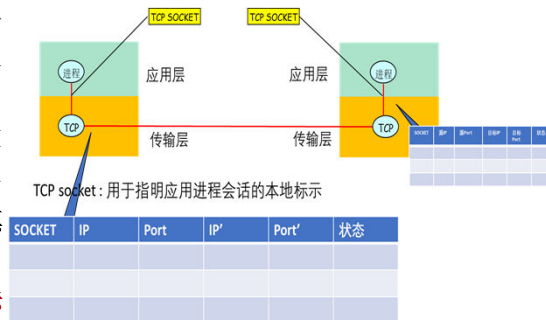


Socket: 14

TCP Socket=(IP,Port,IP',Port')

2025中科大网络

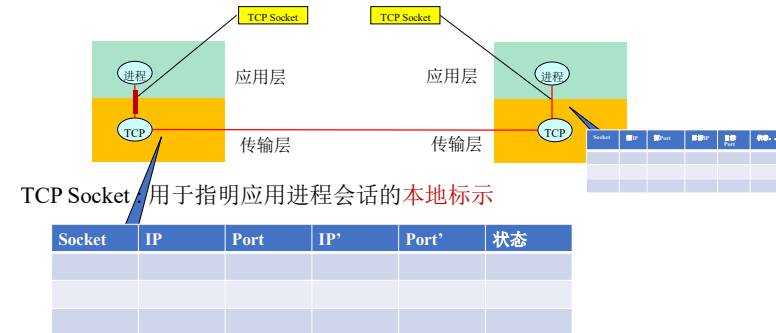
- 在每个端系统OS内各自用一个整数Socket代表2个进程之间的通信关系
- 应用/传输层信息：Socket, M
- 发送方TCP：利用Socket进行TCP段头的**源、目标端口**的填写，**源、目标IP地址**下交
- 接收方根据：**TCP源、目标端口号**和**源、目标IP地址**定位到接收方的Socket 交给应用进程



Socket: 15

TCP Socket: 应用层和OS关于连接标识的约定

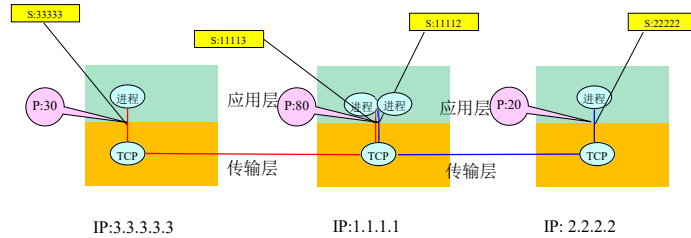
2025中科大网络



Socket: 16

TCP Socket

2025中科大网络



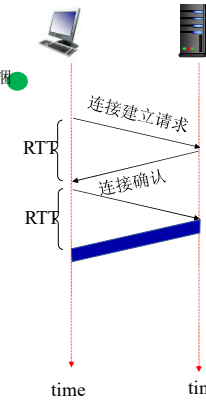
一个TCP Port 上面可以有多个进程

Socket: 17

TCP 连接建立过程

2025中科大网络

1. 客户端运行
2. 创建 C_Socket 3333 和本地某个端口绑定
3. 与服务端建立连接 (connect), 指定服务器的IP和端口号, 阻塞
4. Connect解除阻塞, 状态为建立, Socket 3333 与本地IP、端口号服务器IP和端口号捆绑
5. 通过Socket 发送HTTP请求, 等待接收来自服务器的响应
6. 客户端通过Socket 3333接收到了来自服务器的响应



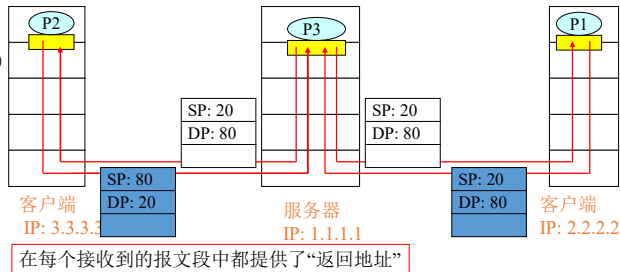
1. 服务进程运行
2. 创建 welcome Socket 1111 与守候的TCP 80号端口捆绑
3. 等待(accept)来自客户端的连接建立
4. Accept 解除阻塞, 返回新的connection Socket 2222, 与本地IP、端口号, 及客户端IP地址和端口号捆绑
5. 通过Socket 2222接收来自客户端的HTTP请求, 响应客户端请求发送响应报文

Application Layer: 2-18

UDP Socket = 本地(IP, Port)

2025中科大网络

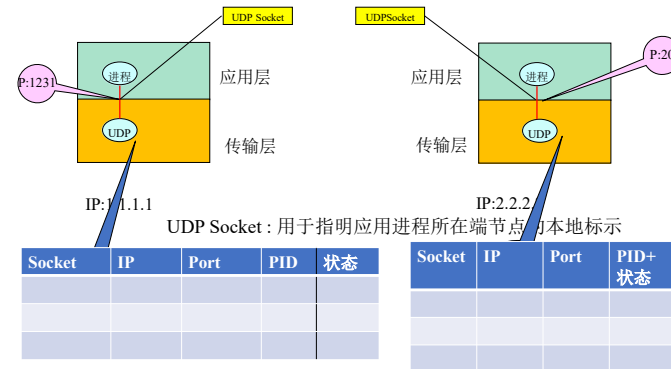
- UDP适合事务性应用
- 无连接 (握手代价大)
- 在报文通信的时候, 通信关系还没建立
- 只能用一个整数 Socket代表通信其中一个本地的端节点
- 穿过层间信息
 - 数据报内容: M
 - 本地端节点: Socket
 - 对方端节点的结构体



Socket: 19

UDP Socket: 应用层和OS关于进程标识的约定

2025中科大网络



Socket: 20

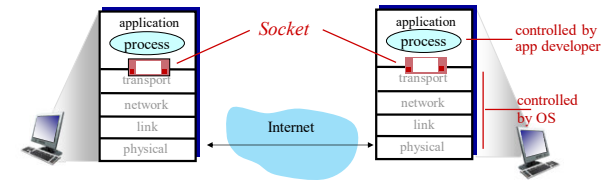
UDP通信过程

-
1. 客户端运转
 2. 创建 C_Socket 999 和本地某个 UDP 端口 90 绑
 3. 利用新创建的 Socket 9999 (对应端口 90) 发送 (send) 数据报, 指明服务器 IP 地址和 80 端口
 4. 从 Socket 9999 接收 (Receive), 等待来自服务器的数据
 5. 客户端接收解除阻塞, 收到来自服务器的应用报文
1. 服务进程运转
 2. 创建 Server Socket 8888 和捆绑和守候 UDP 80 端口
 3. 接收 (Receive) 来自 Socket 8888 实际上是客户端的数据, 阻塞
 4. 接收 (Receive) 解除阻塞, 利用 8888 Socket (端口 80) 返回数据
 5. 发送 (Send) 利用 Socket 8888 (端口 80) 发送数据 指明对方 IP 地址和 90 端口

Application Layer: 2-21

传输层提供的服务形式: Socket API

- 应用进程使用传输层 (网络基础设施, OS) 提供的服务: **Socket API**
 - Socket 作为 API 的一个参数, 代表编址信息, 本质上就是 SAP
- Socket 类似于一个门
 - 发送进程将报文推出这个门, 接收进程从它对应的门上收到报文
 - 涉及到 2 个 Sockets, 两端各自的 Socket



Application Layer: 2-22

应用层协议定义:

- 应用协议必要性: 应用进程在传输层提供的 Socket API 服务的基础上远程交换报文, 需要定义应用协议
- 报文的类型: 请求、响应
- 报文的语法:
 - 报文有哪些字段 & 字段的字节范围
- 报文语义:
 - 每字段等于什么值代表什么含义
- 时序规则: 什么时候发送 & 响应报文, 怎么发送和响应

开放协议:

- 由 RFC 文档定义, 每个人或机构都可以下载协议标准
- 允许不同厂商的设备互操作
- e.g., HTTP, SMTP

私有协议

- e.g., Skype, Zoom

Application Layer: 2-23

传输层提供的服务质量

可靠性

- 一些应用 (e.g., 文件传输, web 事务) 需要 100% 可靠数据传输
- 其他应用可以容忍一定比例的丢失 (e.g., audio)

延迟 (包括延迟差等与时间相关的因素)

- 一些应用 (e.g., 互联网电话和交互游戏) 需要低延迟, 让用户体验好

吞吐量

- 一些应用 (e.g., 多媒体) 需要传输层提供最小限度的吞吐量让应用能够“有效”运行
- 其他应用 (“弹性应用”) 对吞吐不敏感, 给多少带宽都可以运行 (如带宽小, 则运行效果差一些)

安全性

- 加密, 数据完整性, 可认证性, ...

Application Layer: 2-24

常见应用对传输层服务的需求

2025中科大网络

应用	数据丢失	吞吐量	时间敏感性?
文件传输/下载	不丢失	弹性	no
e-mail	不丢失	弹性	no
Web 文本	不丢失	弹性	no
实时音视频	丢失可容忍	audio: 5Kbps-1Mbps video: 10Kbps-5Mbps	yes, 10's msec
流化音视频	丢失可容忍	同上	yes, few secs
交互式游戏	丢失可容忍	Kbps+	yes, 10's msec
短消息	不丢失	弹性	yes and no

Application Layer: 2-25

互联网的传输层服务

2025中科大网络

TCP 服务:

- 在发送进程和接收进程之间进行可靠数据（字节流）传输
- 流量控制: 发送方不会淹没接收方
- 拥塞控制: 当网络过载时抑制发送方的发送速率
- 面向连接: 发送方进程和接收方在通信前需要建立连接
- 不提供: 延迟、最低吞吐量保障和安全性

UDP 服务:

- 不可靠数据传递: 在发送进程和接收进程间数据传输是不可靠的
- 不提供: 可靠性, 流量控制, 拥塞控制, 时间保证, 吞吐量保证, 也不提供安全性和连接建立

Q: 为什么不直接使用IP提供的服务? UDP存在的理由?

Application Layer: 2-26

互联网应用使用的传输层协议

2025中科大网络

应用	应用层协议	传输层协议
文件传输/下载	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web 文件	HTTP 1.1 [RFC 7320]	TCP
互联网电话	SIP [RFC 3261], RTP [RFC 3550], or	TCP or UDP
音视频流化	proprietary HTTP [RFC 7320], DASH	TCP
交互游戏	WOW, FPS (proprietary)	UDP or TCP

Application Layer: 2-27

提升TCP的安全性

2025中科大网络

普通TCP和UDP套接字:

- 没有加密
- 甚至通过Socket发送的密码都以明文的形式传输 (!)

Transport Layer Security (TLS)

- 提供加密的TCP连接服务
- 数据完整性
- 端节点的认证

TLS在应用层实现

- 应用使用TLS库（在应用层），TLS使用TCP
- 通过TLS Socket传输的数据在互联网上是加密的
- 第8章（网络安全）详解

Application Layer: 2-28

应用层

2025中科大网络

- 应用层协议原理
- Web和HTTP**
- E-mail, SMTP, IMAP
- DNS (The Domain Name System)
- P2P 应用
- 视频流化和CDN (content distribution networks)
- UDP和TCP Socket 编程



Application Layer: 2-29

Web 和 HTTP

2025中科大网络

首先, 快速浏览一下web和HTTP协议...

- Web网页包含了一些**对象**, 每个对象可以被存储在不同的(也可以相同的)Web服务器上
- 对象可以是HTML文件, JPEG图像, Java applet, 音频文件, ...
- Web网页包含一个**base HTML-file (对象)**, 该文件包含了若干个被引用对象的URL地址...

http://user:pw@ www.someschool.edu : port /someDept/pic.gif

host name path name

Application Layer: 2-30

HTTP 概述

2025中科大网络

HTTP: hypertext transfer protocol

- 支持Web的应用层协议
- Client/Server模式:
 - Client:** 浏览器, 请求(使用HTTP协议)和接收对象, 解释和“显示”对象(该步骤与协议无关, 属于Web应用中的HTML格式定义)
 - Server:** Web服务器响应Web浏览器的请求, 用HTTP报文给出回应



Application Layer: 2-31

HTTP 概述 (续)

2025中科大网络

HTTP 采用TCP协议:

- 客户端初始化到服务器80号端口的TCP连接 (创建Socket)
- 服务器接收来自客户端的TCP连接建立请求
- HTTP报文 (应用层协议数据单元) 在浏览器 (HTTP客户端) 和服务器的 (HTTP服务器) 之间交换
- 关闭TCP 连接

HTTP 是“无状态的 (stateless)”

- 服务器不维护客户端的状态, 不知道该客户端是否请求过某些对象

—aside—

维护“状态”的协议较复杂!

- 过去的历史 (状态) 需要被维护
- 如果S/C宕机, 它们各自有关于“状态”的信息可能是不一致的, 必须有同步的过程

Application Layer: 2-32

HTTP 连接: 2种类型

非持久HTTP连接

1. 建立TCP连接
2. 在该TCP连接上传送一个对象
3. 关闭TCP连接

请求多个对象需要多个TCP连接

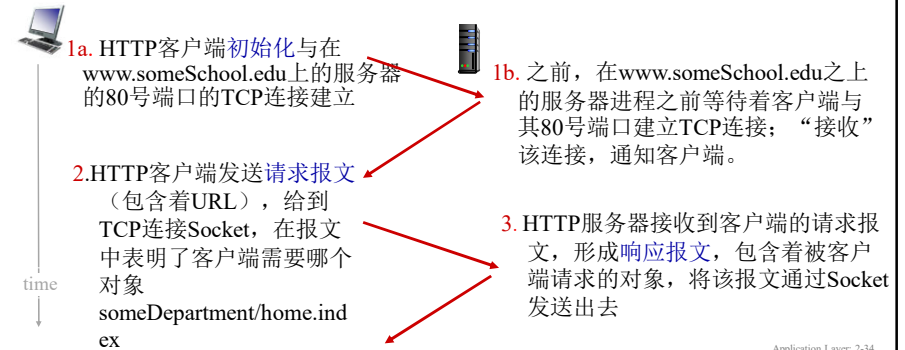
持久HTTP连接

- 建立起来和服务器的TCP连接
- 在该TCP连接上可以传输多个对象
- 关闭TCP连接

Application Layer: 2-33

非持久HTTP: 例子

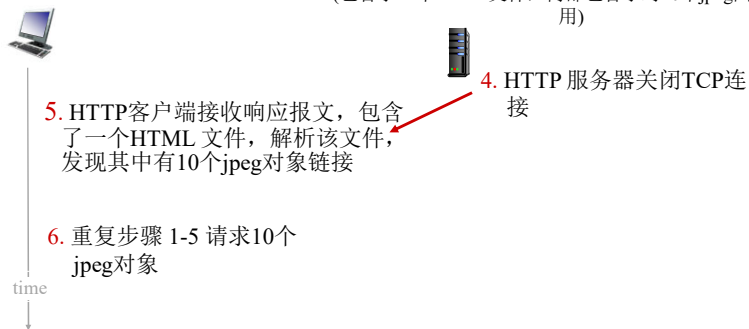
用户键入URL: `www.someSchool.edu/someDepartment/home.index`
(包含了一个HTML文件, 内部包含了对10个jpeg图像的引用)



Application Layer: 2-34

非持久HTTP: 例子 (续)

用户键入URL: `www.someSchool.edu/someDepartment/home.index`
(包含了一个HTML文件, 内部包含了对10个jpeg图像的引用)



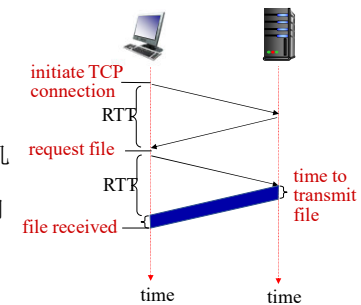
Application Layer: 2-35

非持久HTTP: 响应时间

RTT (定义): 从请求一个小的对象到服务器返回的时间

HTTP 响应时间 (每个对象):

- 一个RTT初始化TCP连接
- 一个RTT HTTP请求到HTTP响应的前几个字节返回
- 还需要加上一个对象/文件的传输时间



非持久HTTP响应时间=2RTT+ 对象传输时间 (每个对象)

Application Layer: 2-36

持久HTTP (HTTP 1.1)

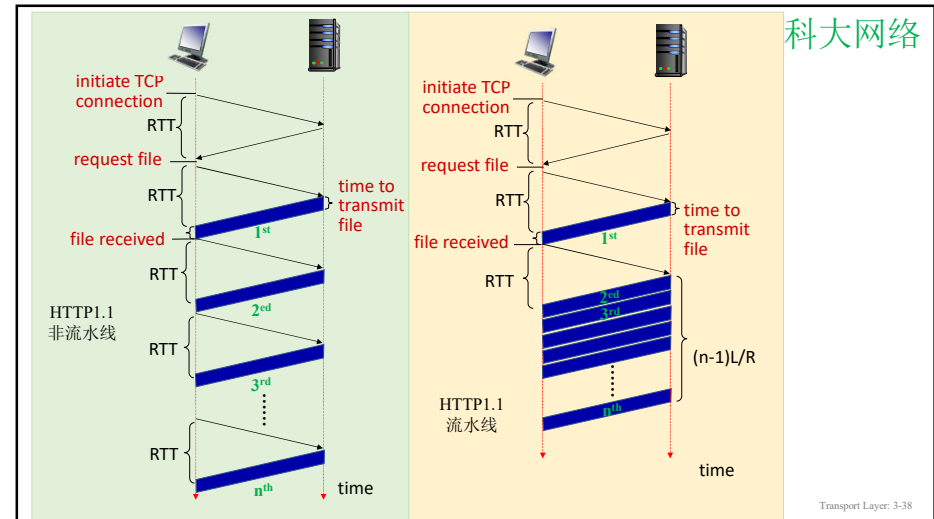
非持久HTTP的问题:

- 每个对象都需要2RTT时间
- 每个TCP连接需要OS的开销
- 浏览器通常会打开和服务器的若干个并行TCP连接, 来获取一个HTML文件所包含的多个对象

持久 HTTP (HTTP1.1):

- 服务器在传输完一个对象之后, 仍然保留着TCP连接
- 后续的HTTP响应报文(被请求的对象在一个服务器之上)在该TCP连接上陆续传输
- 流水线方式: 客户端在遇到每个被引用的对象时立即在TCP连接上发出请求
- 流水线方式: 所有的被引用对象会在一个RTT时间内被获取(减少一半的响应时间)

Application Layer: 2-37



Transport Layer: 3-38

HTTP请求报文

- 2种类型的HTTP报文: 请求request, 响应response
- HTTP 请求报文:
 - ASCII (人类可读格式)

请求行 (GET, POST, HEAD 命令)

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0)
Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

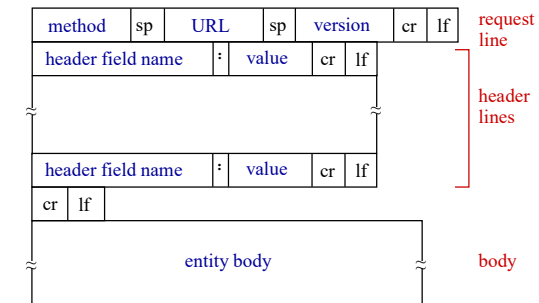
回车符
换行符

回车, 之后的一个空行
代表着头部行的结束
carriage return,

* Check out the online interactive exercises for more examples:
http://gaia.cs.umass.edu/kurose_ross/interactive/

Application Layer: 2-39

HTTP 请求报文: 通用格式



Application Layer: 2-40

其他的HTTP请求报文

POST方法:

- Web网页通常包含有表单输入
- 包含在HTTP POST请求报文的实体中的用户输入从客户端发向服务器

GET方法 (用于发送数据到服务器上):

- 在HTTP GET请求消息的URL字段中包括着用户数据 (后面跟着一个“?”)

www.somesite.com/animalsearch?monkeys&banana

HEAD 方法:

- 仅仅请求URL对象的头部, 而HTTP GET方法中会请求URL对象的全部

PUT方法:

- 向服务器上载文件 (对象)
- 用PUT HTTP请求消息实体中的内容完全替换指定URL处存储的文件

Application Layer: 2-41

HTTP 响应报文

状态行 (状态码
状态码的解释)

头部行

数据, e.g., 所请求的HTML
文件

```
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips
  PHP/7.4.9 mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n
data data data data data ...
```

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Application Layer: 2-42

HTTP 响应状态码

- 状态代码出现在响应报文的第一行: 服务器到客户端的响应报文
- 一些样例代码:

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

Application Layer: 2-43

模拟web客户端尝试一下

1. netcat 到某个你喜爱的Web服务器

```
% nc -c -v gaia.cs.umass.edu 80
```

- 打开和到gaia.cs.umass.edu服务器80号(默认的HTTP服务端口的TCP连接
- 任何输入的字符都会被发送到gaia.cs.umass.edu服务器的80号端口

2. 键入GET HTTP 请求报文到服务器:

```
GET /kurose_ross/interactive/index.php HTTP/1.1
```

```
Host: gaia.cs.umass.edu
```

- 通过键入样例文字 (需要敲击回车两次), 可以将一个最小的 (但完整) GET请求发送到HTTP服务器

3. 看下HTTP服务器发送的响应报文

(或者使用Wireshark软件查看所捕获的HTTP请求/响应)

Application Layer: 2-44

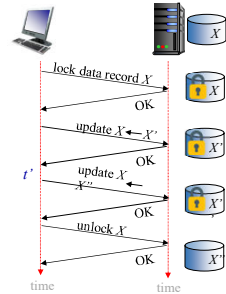
维护客户端/服务器状态: cookies

2025中科大网络

回顾: HTTP请求/响应交互是 **无状态的**

- 没有多步HTTP信息交换来完成Web事务的概念
 - 没有必要让客户端/服务器追踪多步信息交换的状态
 - 所有的HTTP请求都是相互独立的
 - 没有必要让客户端/服务器从一个 **部分完成但是永远无法完成的事务** 中恢复出来

一个有状态的协议: 客户端针对X修改了两次, 或者根本没改



Q: 如果网络连接或客户端在t崩溃会发生什么?

Application Layer: 2-45

维护客户端/服务器状态: cookies

2025中科大网络

Web服务站点和客户端浏览器使用 **cookies** 在它们之间的事务上维持状态

四个组成部分:

- 1) 包含在响应报文的头部行中Cookie
- 2) 包含在请求报文头部行中的Cookie
- 3) Cookie文件被保存在用户的端系统中, 被用户的浏览器管理
- 4) 在Web服务站点的后台数据库中

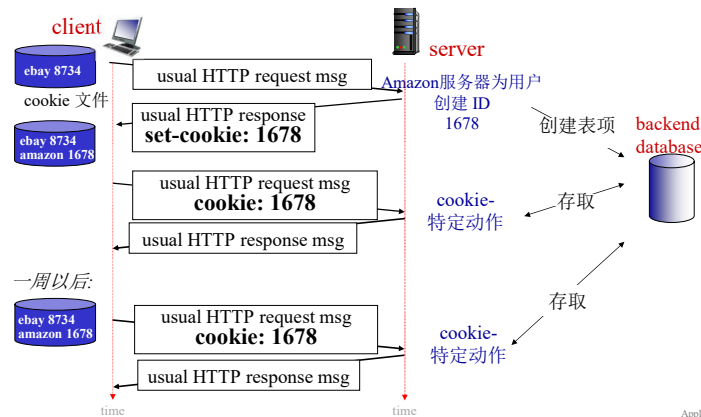
例子:

- Susan使用笔记本浏览器, 第一次访问特定电子商务网站
- 当出第一个HTTP请求报文到达服务器, 服务器站点创建了:
 - 唯一的ID (aka“cookie”)
 - 后台数据库存储了该ID条目
- 后续的来自Susan到达该网站的HTTP请求, 都会包含这个cookie ID值, 允许网站“标识” Susan

Application Layer: 2-46

维护客户端/服务器状态: cookies

2025中科大网络



Application Layer: 2-47

HTTP cookies: 评点

2025中科大网络

Cookies 可以干什么:

- 认证
- 购物车
- 推荐
- 用户会话管理(Web e-mail)

挑战: 如何维护状态?

- 在协议端节点: 在发送端/接收端的多步事务上维护状态
- 在报文内部: HTTP报文携带的cookies

Cookies 和隐私: aside

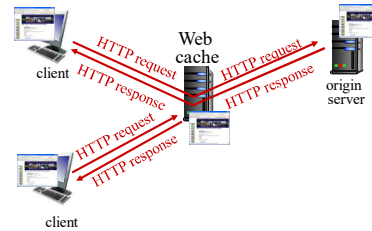
- Cookies可以让服务站点获取很多客户端的用户信息.
- 第三方的持久cookies (追踪cookies) 允许在多个web站点上追踪同一个人的身份?

Application Layer: 2-48

Web 缓存

Goal: 满足用户的请求，但是不会访问原始服务器

- 用户配置其浏览器来指定一个本地的**Web 缓存**
- 浏览器将HTTP请求发向缓存
 - 如果被请求对象在缓存中：缓存直接返回给客户端被请求的对象
 - 否则：向原始服务器请求对象，缓存接收并缓存下来，将该对象返回给客户端



Application Layer: 2-49

Web 缓存 (aka 代理服务器)

- Web缓存表现的既是客户端又是服务器
 - 对于请求对象的客户端来说是服务器
 - 对于原始服务器来说是客户端
- 服务器在响应报文的头部告诉缓存，关于该对象是否允许被缓存

Cache-Control: max-age=<seconds>

Cache-Control: no-cache

为什么Web缓存?

- 降低了客户端请求的响应时间
 - 缓存更加靠近客户端
- 降低了机构接入链路的流量
- 减轻了服务器的负担
- Internet部署了很多缓存
 - 可以让服务能力比较弱的ICP，也可以进行高效的内容分发

Application Layer: 2-50

缓存的例子

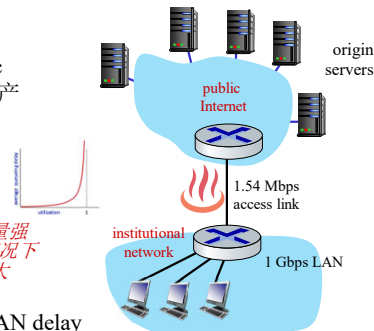
场景:

- 接入链路带宽: 1.54 Mbps
- 从机构的路由器到服务器的RTT: 2 sec
- web 对象大小是100Kbits, 机构浏览器产生的平均请求对象速率是: 15次/sec
 - 到浏览器平均速率为: 1.50 Mbps

性能:

- 接入链路的利用率 = .97
- LAN 利用率: .0015
- 端到端延迟 = Internet delay + access link delay + LAN delay = 2 sec + minutes + usecs

问题: 在流量强度接近于1情况下排队延迟很大



Application Layer: 2-51

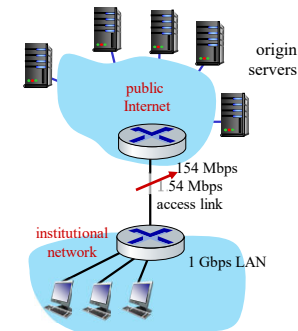
升级选项1: 购买更快的接入链路

场景:

- 接入链路带宽: 154 Mbps
- 机构路由器到服务器的RTT: 2 sec
- Web对象大小: 100K bits
- 从机构浏览器汇总向服务器的平均对象请求速率: 15次/sec
 - 到达浏览器的平均速率: 1.50 Mbps

性能:

- 接入链路利用率 = .0097
- LAN 利用率: .0015
- 端到端延迟 = Internet delay + access link delay + LAN delay = 2 sec + minutes + usecs



代价: 更快的接入链路意味着更多的花费

Application Layer: 2-52

升级选项2: 安装web缓存

2025中科大网络

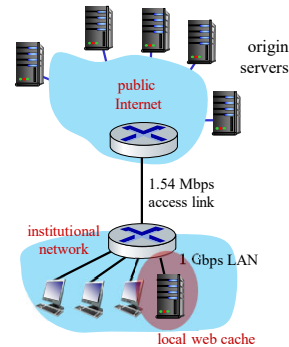
场景:

- 接入链路带宽: 1.54 Mbps
- 机构路由器到服务器的RTT: 2 sec
- Web对象大小: 100K bits
- 从机构浏览器汇总向服务器的平均对象请求速率: 15次/sec
 - 到达浏览器的平均速率: 1.50 Mbps

代价: web缓存 (廉价解决方案) !

性能:

- LAN 利用率: .?
 - 接入链路利用率 = ?
 - 平均端到端延迟 = ?
- 如何计算链路利用率, 延迟?



Application Layer: 2-53

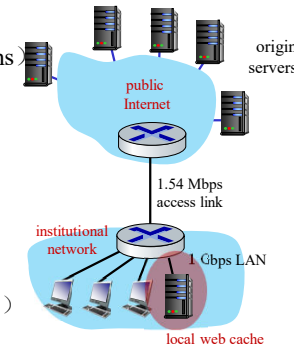
计算缓存条件下的接入链路利用率和端到端延迟

2025中科大网络

假设缓存命中率是 0.4:

- 40% 的请求被缓存满足, 具有很低延迟 (ms)
- 60% 请求被原始服务器满足
 - 经过接入链路到达浏览器的速率
 $= 0.6 * 1.50 \text{ Mbps} = 0.9 \text{ Mbps}$
 - 接入链路利用率 $= 0.9 / 1.54 = .58$ 意味着在接入链路这块排队延迟很低(msec)
- 平均端到端延迟:
 - $= 0.6 * (\text{对象来自原始服务器的延迟})$
 - $+ 0.4 * (\text{在缓存被满足的对象访问延迟})$
 - $= 0.6 (2.01) + 0.4 (\sim \text{msecs}) = \sim 1.2 \text{ secs}$

相对于15Mbps解决方案而言其端到端延迟较低 (而且方案比较廉价)



Application Layer: 2-54

条件式GET

2025中科大网络

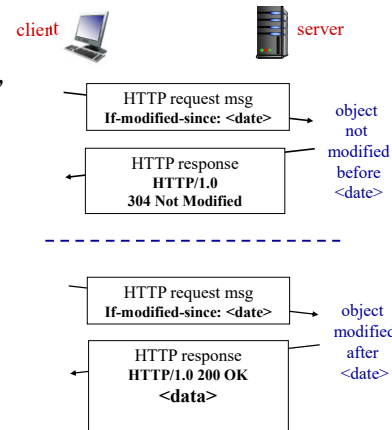
目标: 如果缓存侧具有最新的对象, 无需访问服务器

- 没有对象的传输延迟 (也无需使用网络资源)

客户端: 在HTTP请求报文的头部指定上次缓存对象的更新日期
If-modified-since: <date>

服务器: 如果在客户端的缓存副本是最新的, 则在响应报文中无需包含对象

HTTP/1.0 304 Not Modified



Application Layer: 2-55

HTTP/2

2025中科大网络

主要目标: 减少多个对象HTTP请求的延迟

HTTP/1.1: 在HTTP1.0的基础上, 在一个TCP连接引入了多个、流水线的GET

- 服务器按序响应这些请求 (FCFS: first-come-first-served scheduling)
- 通过FCFS, 一些小的对象可能不得不等待 (HOL 阻塞: head-of-line blocking) 大对象的传输完毕才能够传输
- 丢失恢复 (重传丢失的TCP段) 时实际上暂停了对对象的传输

Application Layer: 2-56

HTTP/2

2025中科大网络

关键目标:减少多个对象HTTP请求的延迟

HTTP/2: [RFC 7540, 2015] 在服务器端增加了向客户端发送对象的灵活性:

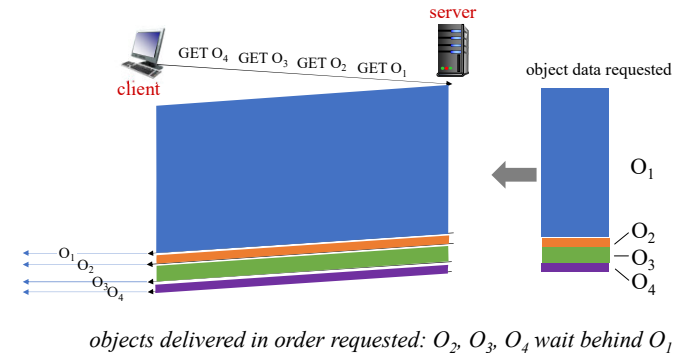
- 相同之处: 方法, 状态码, 大部分头部字段都和HTTP1.1无异
- 通过流号区分各个请求的对象, 将TCP连接进一步复用, 将一个流内传输的对象分成帧
- 被请求对象的传输次序基于客户端指定的**对象优先级** (不见得是FCFS), 调度这些frames的传输次序来缓解HOL阻塞
- 服务器可以主动向客户端推送一些它可能需要的对象 (没请求的)
- 二进制编码
- 头部压缩: 采用HPACK技术减少头部开销, 同样的头部用编码表示

Application Layer: 2-57

HTTP/2: 减轻HOL阻塞

2025中科大网络

HTTP 1.1: 客户端请求1个大对象 (例如一个视频文件, 以及3个小对象

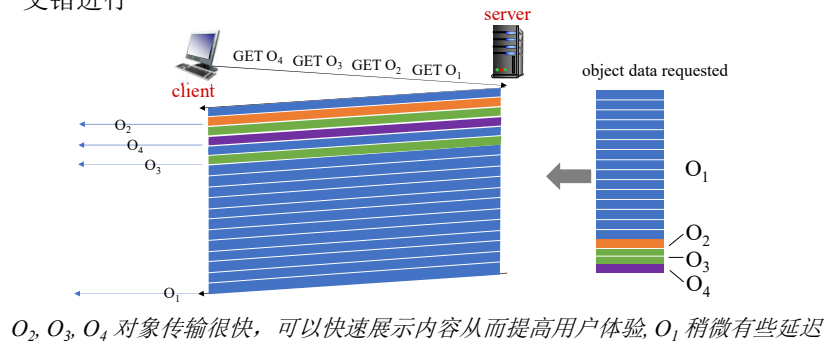


Application Layer: 2-58

HTTP/2: 减轻HOL阻塞

2025中科大网络

HTTP/2: 被请求的对象在服务器那里被分为frames, frames的传输可以交错进行



Application Layer: 2-59

HTTP/2 到 HTTP/3

2025中科大网络

HTTP/2 在一个TCP连接上, 意味着:

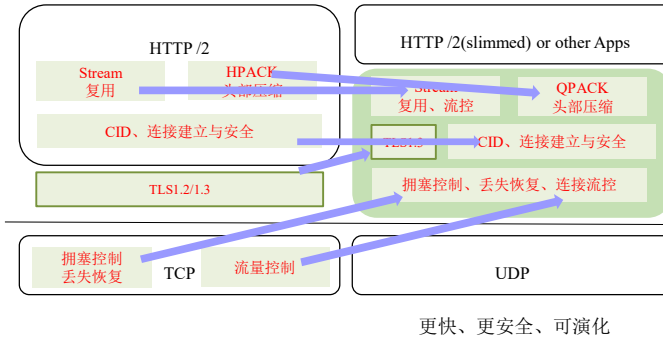
- 恢复一个段的丢失, 需要停止所有后续对象的传输
 - 和HTTP1.1一样, 浏览器可以同时打开多个并行的连接来减少这种停顿, 增加总体的吞吐量
 - 但是服务器和客户端的TCP连接建立个数有限 (默认4个)
- HTTP/2 over 基本TCP连接上, 并没有增加其安全性
 - http over TLS 协议层次多, 总体效率低
- **HTTP/3:** 提供安全性, 在UDP之上实现了对象差错控制和拥塞控制 (通过多个并行quic连接传输, 提高效率)
 - QUIC和TLS 协作关系提升效率, HTTP/3 详见传输层

Application Layer: 2-60

HTTP/3

2025中科大网络

■ HTTP/3=HTTP/2+QUIC

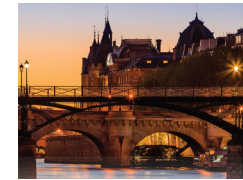


Application Layer: 2-61

应用层

2025中科大网络

- 应用层协议原理
- Web和HTTP
- E-mail, SMTP, IMAP
- DNS (The Domain Name System)
- P2P 应用
- 视频流化和CDN (content distribution networks)
- UDP和TCP Socket 编程



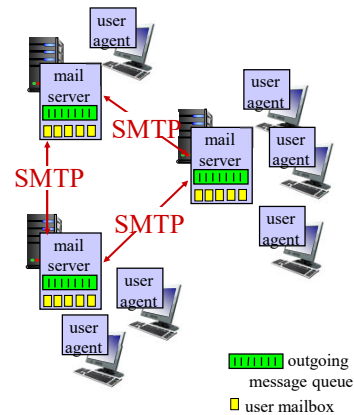
Application Layer: 2-62

E-mail

2025中科大网络

3个主要构成器件:

- 用户代理
- 邮件服务器
- 简单邮件传输协议: SMTP

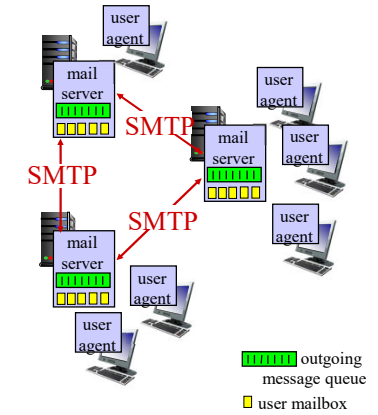


E-mail

2025中科大网络

用户代理

- a.k.a. 邮件阅读器 (编辑器)
- 撰写、编辑和阅读邮件
- e.g., Outlook, iPhone mail client
- 发送/接收的邮件被存储在服务器



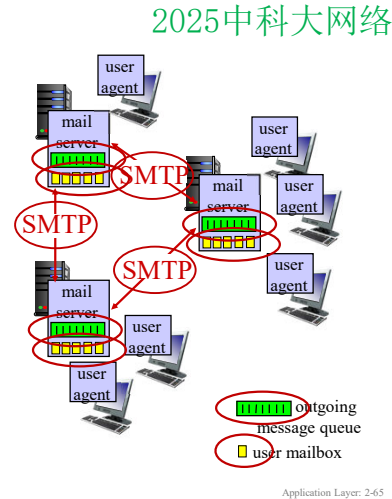
E-mail: 邮件服务器

邮件服务器:

- 收件箱 *mailbox* 包括了到其用户的邮件
- 邮件队列 *message queue* 包括了将要发出的邮件

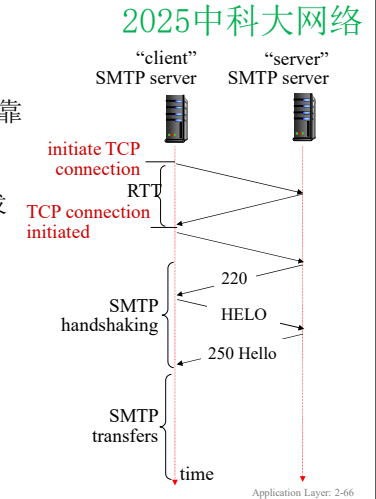
SMTP协议在邮件服务器之间发送邮件的协议

- 客户端: 发送邮件的服务器
- 服务器: 接收邮件的服务器



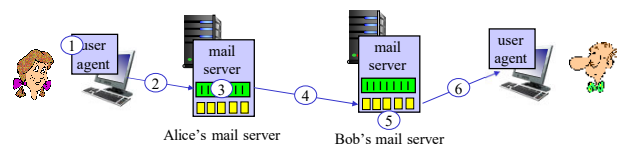
SMTP RFC (5321)

- 在客户端和服务器之间采用TCP进行可靠的邮件传输（邮件服务器初始化连），服务器端口25
- 直接传输: 源邮件服务器（客户端）直接发送到接收邮件服务器
- 传输邮件的3个阶段
 - SMTP握手（互致问候）
 - SMTP传输邮件阶段
 - SMTP关闭
- 命令/响应交互（类似于HTTP）
 - 命令: ASCII 字符
 - 响应: 状态码和状态码的解释



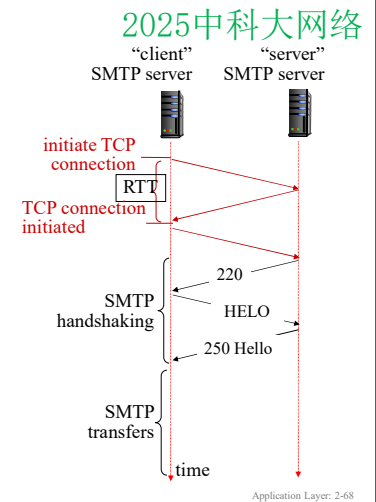
例子: Alice发送一封电子邮件给Bob

- 1) Alice使用用户带来撰写e-mail邮件给 bob@someschool.edu
- 2) Alice的邮件代理采用SMTP协议发送邮件给自己的邮件服务器，邮件被放置在邮件队列中
- 3) 邮件客户端打开到接收邮件的邮件服务器（Bob的邮件服务器）的TCP连接
- 4) SMTP客户端通过TCP连接发送Alice的邮件
- 5) Bob的邮件服务器将邮件放到Bob的邮箱
- 6) Bob运行他的用户代理阅读邮件



Sample SMTP interaction

S: 220 hamburger.edu
 C: HELO crepes.fr
 S: 250 Hello crepes.fr, pleased to meet you
 C: MAIL FROM: <alice@crepes.fr>
 S: 250 alice@crepes.fr... Sender ok
 C: RCPT TO: <bob@hamburger.edu>
 S: 250 bob@hamburger.edu ... Recipient ok
 C: DATA
 S: 354 Enter mail, end with "." on a line by itself
 C: Do you like ketchup?
 C: How about pickles?
 C: .
 S: 250 Message accepted for delivery
 C: QUIT
 S: 221 hamburger.edu closing connection



SMTP: 观察

2025中科大网络

与HTTP对比:

- HTTP: client拉
- SMTP: client推
- 二者都采用ASCII格式的命令和响应, 状态码
- HTTP: 每个对象封装在各自的响应报文中
- SMTP: 多个邮件内的对象被包含在一个邮件中 (有格式的多组成部分)
- SMTP使用持久连接
- SMTP服务器使用CRLF.CRLF决定报文的结束
 - TCP提供字节流的服务, 不提供报文之间的界限
 - 应用自己确定报文之间的界限)

Application Layer: 2-69

邮件报文格式

2025中科大网络

SMTP: 交换电子邮件的协议, RFC 5321 (HTTP RFC 7231)

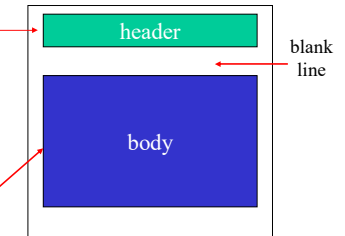
RFC 2822 定义了电子邮件本身的语法 (就像HTML定义了web文档的语法一样)

▪ 头部行, e.g.,

- To:
- From:
- Subject:

这些行, 在电子邮件区域的正文中, 不同于SMTP MAIL from: 和RCPT TO:等命令!

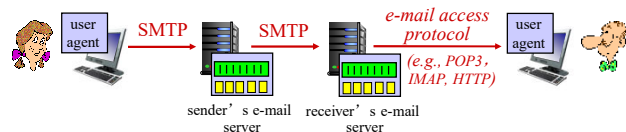
▪ 邮件的正文: 报文内容, 仅限于用ASCII字符描述



Application Layer: 2-70

访问邮件: 邮件访问协议

2025中科大网络



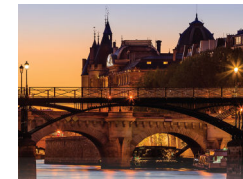
- **SMTP**: 传送并存储到接收方邮件服务器中
- 邮件访问协议: 接收方从自己的邮件服务器那里访问发送给他的邮件
 - **IMAP**: Internet Mail Access Protocol [RFC 3501]: 邮件存储在服务器上, IMAP 协议提供在服务器上读取, 删除邮件的功能, 并且支持管理目录
- **HTTP**: gmail, Hotmail, Yahoo!Mail, etc.基于web的访问接口, 在SMTP 协议之上发送邮件, 或在IMAP(POP)之上收取邮件
 - HTTP over (SMTP and IMAP)

Application Layer: 2-71

应用层

2025中科大网络

- 应用层协议原理
- Web和HTTP
- E-mail, SMTP, IMAP
- **DNS (The Domain Name System)**
- P2P 应用
- 视频流化和CDN (content distribution networks)
- UDP和TCP Socket 编程



Application Layer: 2-72

2025中科大网络

DNS的必要性

- 通过URL进行Web访问和通过邮件地址发送邮件都需要服务器（或主机）地址
- 一种方法是直接使用服务器的IP地址
- 但IP地址不好记忆，不便人类使用(没有意义)
- 人类一般倾向于使用一些有意义的字符串来标识Internet上的设备
 - 例如：访问 <https://www.ustc.edu.cn/index.html> 所在的web服务器
 - qzheng@ustc.edu.cn 所在邮件服务器
- 存在着“字符串”——IP地址转换的必要性
- 人类用户提供要访问机器的“字符串”名称
- 由DNS负责转换成为IP网络地址

Application Layer: 2-73

2025中科大网络

DNS系统需要解决的问题

- 问题1：如何命名设备
 - 用有意义的字符串：好记，便于人类用使用
 - 解决一个平面命名的重名问题：层次化命名
- 问题2：如何完成名字到IP地址的转换
 - 分布式的数据库维护和响应名字查询
- 问题3：如何维护
 - 增加（或删除）一个域，需要在域名系统中做哪些工作
 - 上层域名服务器给出子域名字服务器的指针
 - 子域的名字是啥，名字服务器的名字
 - 名字服务器的名字，对应的IP地址

Application Layer: 2-74

2025中科大网络

DNS(Domain Name System)的历史

- ARPANET的名字解析解决方案
 - 主机名：没有层次的一个字符串（一个平面）
 - 存在着一个（集中）维护站：维护着一张 主机名-IP地址的映射文件：Hosts.txt
 - 每台主机定时从维护站取文件
 - ARPANET解决方案的问题
 - 当网络中主机数量很大时
 - 没有层次的主机名称很难分配
 - 文件的维护、发布、查找都很麻烦
- A: 不可扩展! 即使是层次化分布化之后*
- 仅Comcast DNS服务器：600B DNS查询/天
 - 仅Akamai DNS 服务器：2.2T DNS 查询/天

Application Layer: 2-75

2025中科大网络

DNS(Domain Name System)总体思路和目标

- DNS总体思路
 - 分层、基于域的命名机制
 - 若干分布式的数据库完成名字到IP地址的转换
 - 运行在UDP之上端口号为53的应用服务
 - 核心的Internet功能，但以应用层协议实现
 - 在网络边缘处理复杂性
- DNS目标：
 - 实现主机名-IP地址的转换 (name/IP translate)
 - 其它目的
 - 主机别名到规范名字的转换：Host aliasing
 - 邮件服务器别名到邮件服务器的正规名字的转换：Mail server aliasing
 - qzheng@ustc.edu.cn 中ustc.edu.cn, 对应的邮件服务器正规名字
 - 负载均衡：Load Distribution

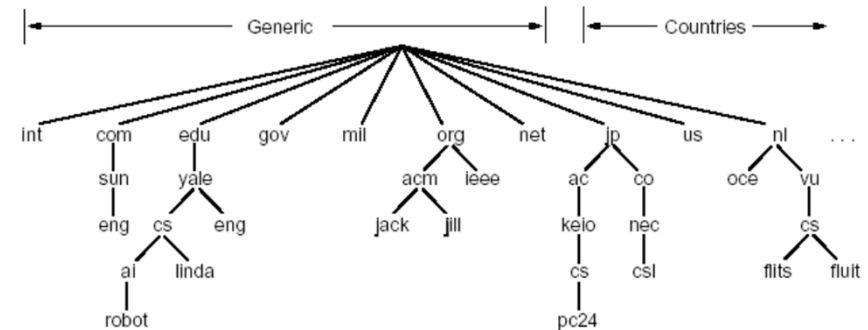
Application Layer: 2-76

问题1: DNS名字空间(The DNS Name Space)

- DNS域名结构
 - 一个层次命名会有很多重名
 - 采用树状层次结构命名
 - Internet 根被划为几百个顶级域(top level domains)
 - 通用的(generic)
 - .com; .edu; .gov; .int;
 - .mil; .net; .org
 - .firm; .hsop; .web
 - ; .arts; .rec;
 - 国家的(countries)
 - .cn; .us; .nl; .jp
 - 每个(子)域下面可划分为若干子域(subdomains)
 - 树叶是主机

Application Layer: 2-77

DNS名字空间(The DNS Name Space)



Application Layer: 2-78

DNS名字空间(The DNS Name Space)

- 域名(Domain Name)
 - 从本域往上, 直到树根
 - 中间使用 “.” 间隔不同的级别
 - 例如:
 - ustc.edu.cn
 - auto.ustc.edu.cn
 - www.auto.ustc.edu.cn
- 域的域名: 可以用于表示一个域
- 主机的域名: 一个域上的一台主机

Application Layer: 2-79

DNS名字空间(The DNS Name Space)

- 域名的管理
 - 一个域管理其下的子域
 - .jp 被划分为 ac.jp co.jp
 - .cn 被划分为 edu.cn com.cn
 - 创建一个新的域, 必须征得它所属域的同意
- 域是逻辑的, 与所在物理网络并不是一一对应的
 - 域遵从组织界限, 是逻辑的, 而不是物理的
 - 一个域的主机可以不在一个IP网络
 - 一个IP网络的主机不一定在一个域

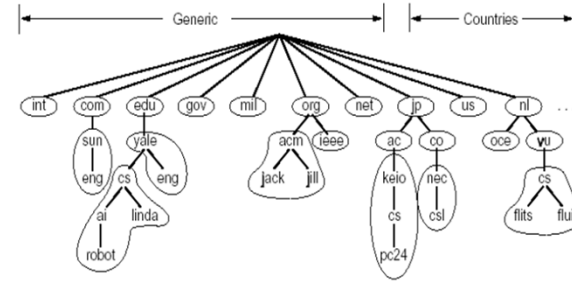
Application Layer: 2-80

问题2：解析问题-名字服务器(Name Server)

- 理论上可以设置一台服务器，存储全球所有域名和IP地址的对应关系
- 一个名字服务器的问题
 - 可靠性问题：单点故障
 - 扩展性问题：通信容量
 - 维护问题：远距离的集中式数据库
- 将DNS名字空间划分为一个个区域(zone)
 - 区域相互不重叠，是树的一部分
 - 区域划分由管理者自己决定
 - 每个区域设置一台名字服务器，将全球繁重DNS解析任务分布化
 - 负责它所管辖区域的名字解析
 - 该名字服务器为该区域的权威名字服务器
 - 名字服务器可以放置在区域之外，以保障可靠性

Application Layer: 2-81

名字空间划分为若干区域：Zone



权威DNS服务器：组织机构的DNS服务器，提供该机构域名与IP之间的映射关系可以选择实现自己维护，或由某个服务提供商来维护

Application Layer: 2-82

区域名字服务器维护资源记录

- 资源记录(resource records)
 - 作用：维护 域名-IP地址 (其它)的映射关系
 - 位置：Name Server的分布式数据库中
- RR格式: (domain_name, ttl, type, class, Value)
 - Domain_name: 域名
 - Ttl: time to live : 生存时间(权威, 缓冲记录)
 - Class 类别：对于Internet，值为IN
 - Value 值：可以是数字，域名或ASCII串
 - Type 类别：资源记录的类型—见下页

Application Layer: 2-83

DNS records

DNS: 保存资源记录(RR)的分布式数据库

RR 格式: (name, value, type, TTL)

type=A

- Name为主机
- Value为IP地址

type=NS

- Name域名(如foo.com)
- Value为该域权威名字服务器的主机名字

type=CNAME

- Name : www.ibm.com为别名，其规范名字为 servereast.backup2.ibm.com
- value 是规范名字

type=MX

- 其值为name对应的SMTP邮件服务器的名字

TTL: 生存时间，决定了资源记录应当从缓存中删除的时间

Application Layer: 2-84

DNS资源记录的例子

2025中科大网络

```

; Authoritative data for cs.vu.nl
cs.vu.nl. 86400 IN SOA star boss (952771,7200,7200,2419200,86400)
cs.vu.nl. 86400 IN TXT "Faculteit Wiskunde en Informatica."
cs.vu.nl. 86400 IN TXT "Vrije Universiteit Amsterdam."
cs.vu.nl. 86400 IN MX 1 zephyr.cs.vu.nl.
cs.vu.nl. 86400 IN MX 2 top.cs.vu.nl.

```

```

flits.cs.vu.nl 86400 IN HINFO Sun Unix
flits.cs.vu.nl 86400 IN A 130.37.16.112
flits.cs.vu.nl 86400 IN A 192.31.231.165
flits.cs.vu.nl 86400 IN MX 1 flits.cs.vu.nl.
flits.cs.vu.nl 86400 IN MX 2 zephyr.cs.vu.nl.
flits.cs.vu.nl 86400 IN MX 3 top.cs.vu.nl.
www.cs.vu.nl 86400 IN CNAME star.cs.vu.nl.
ftp.cs.vu.nl 86400 IN CNAME zephyr.cs.vu.nl.

```

```

rowboat IN A 130.37.56.201
IN MX 1 rowboat
IN MX 2 zephyr
IN HINFO Sun Unix

```

```

littie-sister IN A 130.37.62.23
IN HINFO Mac MacOS

```

```

laserjet IN A 192.31.231.216
IN HINFO "HP Laserjet IIIS" Proprietary

```

Name	Value	Class	Type	TTL
flits.cs.vu.nl	192.31.231.165	IN	A	
Rowboat.cs.vu.nl	130.37.56.201	IN	A	
laserjet.cs.vu.nl	192.31.231.261	IN	A	

某区域的名字服务器中的RR

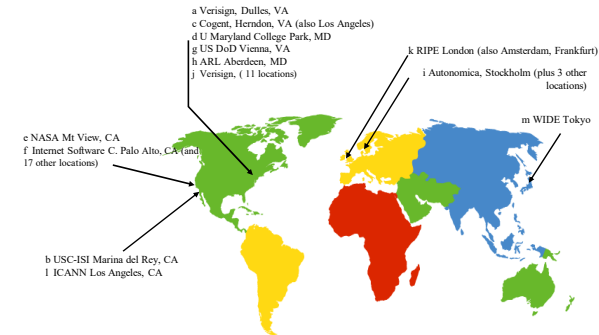
A记录：名字空间和网络空间的映射关系

示意图来自Tanenbaum《计算机网络(第6版)》

85

DNS: 根名字服务器

2025中科大网络

共有13个IPv4逻辑根服务器
固定IP地址

Application Layer: 2-86

查询1：顺着树根往下找到权威名字服务器

2025中科大网络



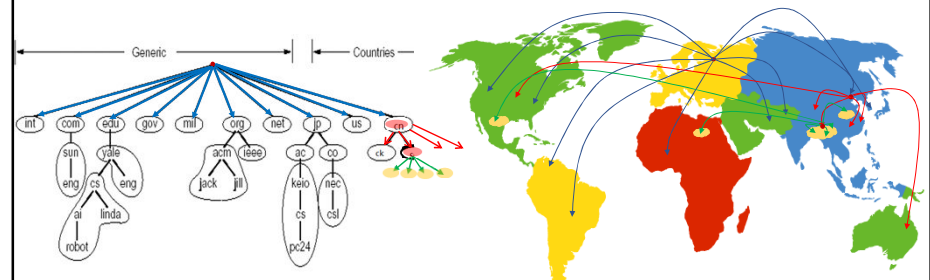
- TYPE NS 下级域名, 配合A类型
- TYPE NS 下级域名服务器IP地址
- TYPE A 本域机器域名的IP地址

纵向的指向：上级域有指向下级域的指针

87

查询2：权威服务器给出域名-IP解析结果

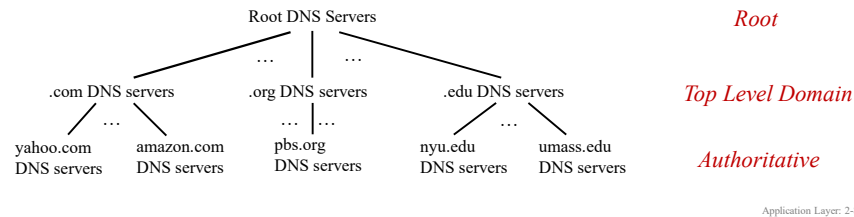
2025中科大网络

横向的指向：权威名字服务器维护本域域名到IP地址的对应关系，
一个区域的设备所在的IP地址不一定在一个IP子网，域是逻辑的，子网是物理的

示意图修改自Tanenbaum《计算机网络(第6版)》以及Kurose《计算机网络-自顶向下方法(第8版)》的ppt

TLD服务器

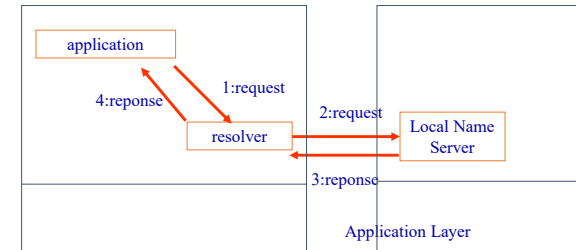
- **顶级域(TLD)服务器**: 负责顶级域名 (如com, org, net, edu和gov) 和所有国家级的顶级域名 (如cn, uk, fr, ca, jp)
 - Network solutions 公司维护com TLD服务器
 - Educause公司维护edu TLD服务器



DNS大致工作过程

如何将解析发到根服务器?

- 应用通过解析器(resolver)向Local Name Server发出查询报文
 - Local Name Server的地址手工配置或者DHCP配置
- Local Name Server返回响应报文(name/ip)



本地名字服务器 (Local Name Server)

- 并不严格属于层次结构
- 每个ISP (居民区的ISP、公司、大学) 都有一个本地DNS服务器
 - 也称为“默认名字服务器”
- 当一个主机发起一个DNS查询时, 查询被送到其本地DNS服务器
 - 起着代理的作用, 将查询转发到DNS层次结构中

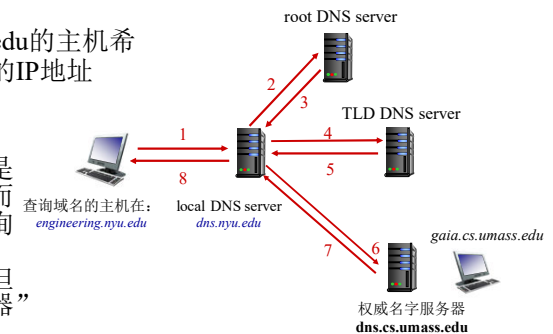
Application Layer: 2-91

DNS 名字解析: 迭代查询

例子: 在engineering.nyu.edu的主机希望获得gaia.cs.umass.edu的IP地址

迭代查询:

- 联系到的服务器并不是直接给出解析结果, 而是返回需要进一步查询的服务器
- “我不知道该名字, 但是你可以问这个服务器”



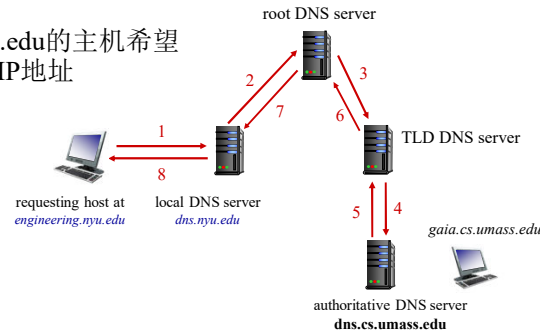
DNS 名字解析: 递归查询

2025中科大网络

例子: 在engineering.nyu.edu的主机希望获得gaia.cs.umass.edu的IP地址

递归查询:

- 将查询任务交给所联系的服务器, 将负担交给该服务器
- 域名层次的上层服务器负担比较重



Application Layer: 2-93

DNS 协议报文

2025中科大网络

DNS **查询**和**应答**报文, 其**格式**一样:

报文头部:

- identification:** 16 bit # 查询标识, 应答应用同样的ID #
- flags:**
 - 查询还是应答
 - 希望递归查询?
 - 递归查询可用?
 - 应答是不是权威的

← 2 bytes →		← 2 bytes →	
identification		flags	
# questions		# answer RRs	
# authority RRs		# additional RRs	
questions (variable # of questions)			
answers (variable # of RRs)			
authority (variable # of RRs)			
additional info (variable # of RRs)			

Application Layer: 2-94

DNS 协议报文

2025中科大网络

DNS **查询**和**应答**报文, 其**格式**一样:

← 2 bytes →		← 2 bytes →	
identification		flags	
# questions		# answer RRs	
# authority RRs		# additional RRs	
questions (variable # of questions)			
answers (variable # of RRs)			
authority (variable # of RRs)			
additional info (variable # of RRs)			

查询报文使用的名字, 类型字段

RRs 针对查询的应答报文

权威名字服务器记录

附加的可能有用信息

Application Layer: 2-95

缓存DNS信息

2025中科大网络

- 任何名字服务器一旦获得一个映射, **缓存**该映射, 如果有相关查询, 它会**立即**给出缓存的映射
 - 缓存提高了响应效率
 - 某缓存表项在超时时间 (TTL) 老化时, 会被删除
 - TLD服务器通常被各本地名字服务器缓存 (无需问根)
- 缓存的表项可能会**过期 (老化)**
 - 如果命名主机更换了IP地址, 可能在缓存TTL到期之前不被互联网范围的设备所知道!
 - 有问题的解析结果是: 访问不成功, 会再次提出**权威记录**的解析要求
 - 尽力而为进行名字到IP地址的转换!**

Application Layer: 2-96

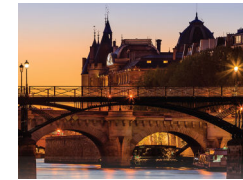
问题3：维护问题：新增一个域

- 在上级域的名字服务器中增加两条记录
 - 新增子域的域名，名字服务，NS
 - 名字服务器，对应的IP地址，A
- 在子域名字服务器上运行名字服务器
 - 新增一些该域常用域名和IP地址的对应关系
- 例：在com域中新增子域“Network Utopia”
 - 到注册登记机构注册域名 networkutopia.com
 - 需要向该机构提供权威DNS服务器（基本的、和辅助的）的名字和IP地址
 - 登记机构在com TLD服务器中插入两条RR记录：
 - networkutopia.com, dns1.networkutopia.com, NS
 - dns1.networkutopia.com, 212.212.212.1, A
 - 在权威服务器212.212.212.1的中增加一些该域的名字和IP地址对应关系
 - 用于Web服务器的www.networkutopia.com 的类型A的记录
 - 用于邮件服务器mail.networkutopia.com的类型MX的记录

Application Layer: 2-97

应用层

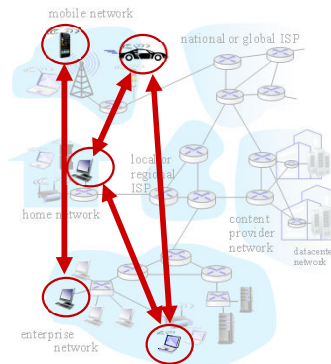
- 应用层协议原理
- Web和HTTP
- E-mail, SMTP, IMAP
- DNS (The Domain Name System)
- P2P 应用
- 视频流化和CDN (content distribution networks)
- UDP和TCP Socket 编程



Application Layer: 2-98

Peer-to-peer (P2P) 架构

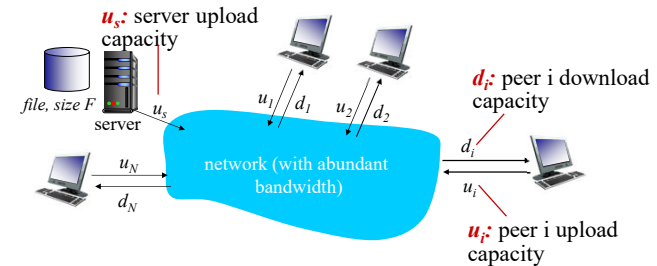
- 没有（或极少）一直运行的服务器
- 任意端系统都可以直接通信
- Peers请求其他peers节点的服务，为其他peers节点提供服务
 - 自扩展-新节点带来新的服务请求，以及新的能力
- Peer节点间歇上网，每次IP地址都有可能变化
 - 管理复杂
- 例子：P2P文件共享(BitTorrent)，流媒体 (KanKan), VoIP (Skype)



Application Layer: 2-99

文件分发: client-server vs P2P

- Q: 对于大小为F的文件，一台服务器分发到N个节点需要耗时多少？
- 节点的上下载能力是受限的资源



Introduction: 1-100

文件分发时间: client-server

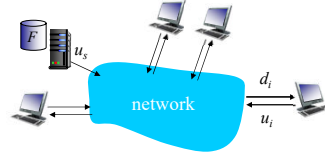
2025中科大网络

- **服务器传输:** 必须顺序传送 (上载)
N份文件副本:

- 传输一份副本需要: F/u_s
- 传输N份副本需要: NF/u_s

- **客户端:** 每个客户端必须下载一份文件副本

- $d_{\min}$ = 客户端下载速率的最小值
- 最少需要耗时进行下载: $F/d_{\min}$



采用C/S方法
分发文件F到N个客户端
最少需要耗时

$$D_{c-s} \geq \max\{NF/u_s, F/d_{\min}\}$$

随着N的增加而线性增加

Introduction: 1-101

文件分发时间: P2P

2025中科大网络

- **服务器传输:** 最少需要上载一份副本:

- 服务器发送一份副本的时间: F/u_s

- **客户端:** 每个客户端最少需要下载一个部分

- 最少需要下载时间: $F/d_{\min}$

- **客户端:** 作为总体必须下载NFbits

- 最大上载带宽是: $u_s + Su_i$
- 除了服务器可以上载, 其他所有的peer节点都可以上载

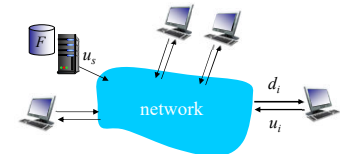
time to distribute F
to N clients using
P2P approach

$$D_{P2P} \geq \max\{F/u_s, F/d_{\min}, NF/(u_s + Su_i)\}$$

分子随着N线性变化, 每个节点需要下载, 整体下载量随着N增大 ...

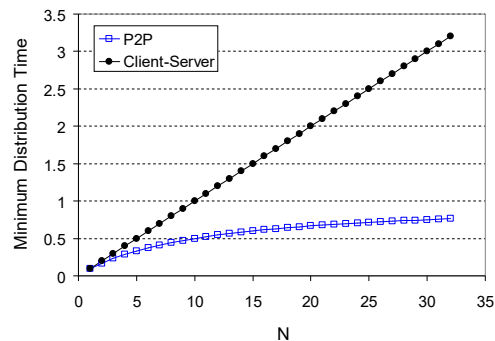
... 分母也是如此, 随着peer节点的增多, 每个peer也带了服务能力

Application Layer: 2-102



Client-server vs. P2P: 例子

2025中科大网络

客户端上载速率 = u , $F/u = 1$ 小时, $u_s = 10u$, $d_{\min} \geq u_s$ 

Application Layer: 2-103

P2P 文件分发: BitTorrent

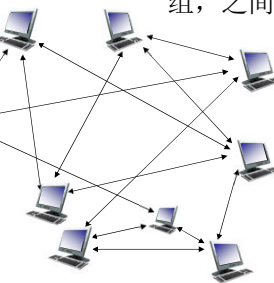
2025中科大网络

- ❖ 文件被分为一个个块256KB

- ❖ 网络中的这些peers发送接收文件块, 相互服务

tracker: 跟踪torrent中
参与节点**Torrent (洪流):** 节点的
组, 之间交换文件块

Alice 加入...

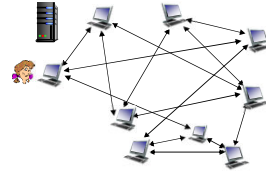
... 从跟踪服务器
处获得peer节点列表... 然后开始和torrent
中的peer节点交换块

Application Layer: 2-104

P2P 文件分发: BitTorrent

2025中科大网络

- Peer加入torrent:
 - 一开始没有块, 但是将会通过其他节点处累积文件块
 - 向跟踪服务器注册, 获得peer节点列表, 和部分peer节点构成邻居关系 (“连接”)
- 当peer下载时, 该peer可以同时向其他节点提供上载服务
- Peer可能会变换用于交换块的peer节点
 - 扰动churn:** peer节点可能上线或者下线
 - 一旦一个peer拥有整个文件, 它会 (自私的) 离开或者保留 (利他主义) 在torrent中



Application Layer: 2-105

BitTorrent: 请求, 发送文件块

2025中科大网络

请求块:

- 在任何给定时间, 不同peer节点拥有一个文件块的子集
- 周期性的, Alice节点向邻居询问他们拥有哪些块的信息
- Alice向peer节点请求它希望的块, 稀缺块优先

发送块: 一报还一报tit-for-tat

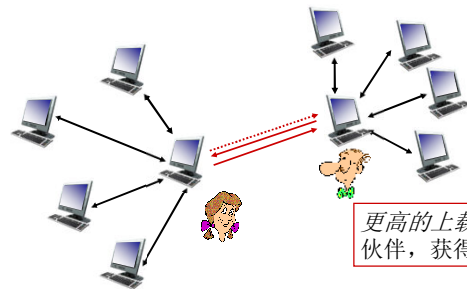
- Alice向4个peer发送块, 这些块向它自己提供最大带宽的服务
 - 其他peer被Alice阻塞 (将不会从Alice处获得服务)
 - 每10秒重新评估一次: 前4位
- 每个30秒: 随机选择其他peer节点, 向这个节点发送块
 - “优化疏通”这个节点
 - 新选择的节点可以加入这个top 4

Application Layer: 2-106

BitTorrent: tit-for-tat

2025中科大网络

- Alice “优化疏通” Bob
- Alice 变成了Bob的前4位提供者; Bob答谢Alice
- Bob 变成了Alice的前4提供者



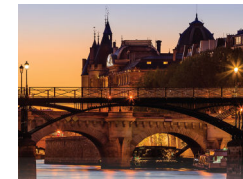
更高的上载速率: 发现更好的交易伙伴, 获得更快的文件传输速率!

Application Layer: 2-107

应用层

2025中科大网络

- 应用层协议原理
- Web和HTTP
- E-mail, SMTP, IMAP
- DNS (The Domain Name System)
- P2P 应用
- 视频流化和CDN (content distribution networks)
- UDP和TCP Socket 编程



Application Layer: 2-108

视频流化服务和CDN：上下文

2025中科大网络

- 视频流量：占据着互联网大部分的带宽
 - Netflix, YouTube, Amazon Prime: 占据80%的住宅ISP流量(2020)
- 挑战1：规模性-如何服务者~1B用户?单个超级服务器无法提供服务（为什么？）
- 挑战2：异构性
 - 不同用户拥有不同的能力（例如：有线接入和移动用户；带宽丰富和受限用户）
 - 需求不一样：大电视和小终端
- 解决：分布式的、应用层面的基础设施

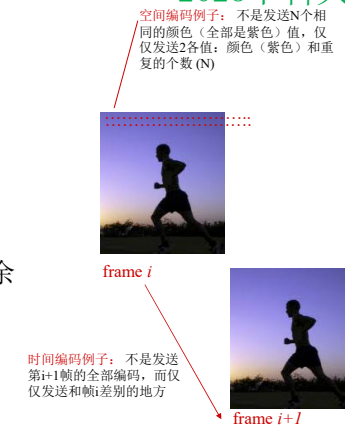


Application Layer: 2-109

多媒体：视频

2025中科大网络

- ❖ 视频：固定速度显示的图像序列
 - e.g., 24 images/sec
- 数字图像：像素的数组
 - 每个像素用若干bits来表示
- 编码：使用图像内和图像间的冗余来降低编码的比特数
 - 空间冗余(图像内)
 - 时间冗余(相邻的图像间)

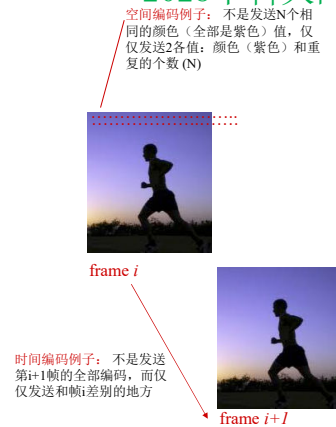


Application Layer: 2-110

多媒体：视频

2025中科大网络

- CBR: (constant bit rate):以固定速率编码
- VBR: (variable bit rate):视频编码率随着空间、时间编码量的变化而变化
- 例子：
 - MPEG 1 (CD-ROM) 1.5 Mbps
 - MPEG2 (DVD) 3-6 Mbps
 - MPEG4 (often used in Internet, 64Kbps – 12 Mbps)
 - AVS

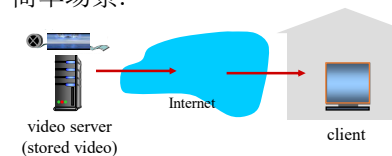


Application Layer: 2-111

存储视频的流化服务：

2025中科大网络

简单场景：

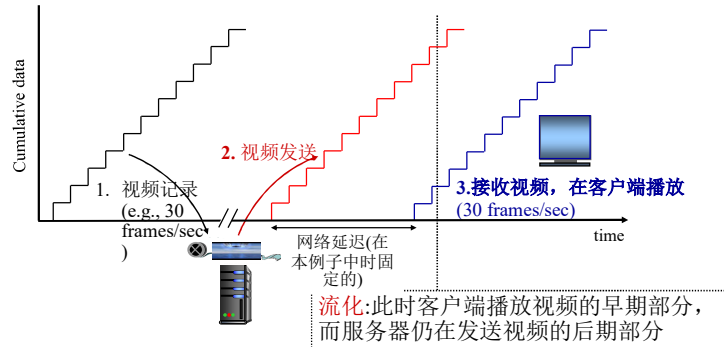


主要挑战：

- server-to-client随着网络拥塞程度而变化，其带宽随时间变化（内部、接入网络、网络核心、视频服务器）
- 数据包丢失、拥塞导致的延迟会播放暂停或视频质量下降

Application Layer: 2-112

存储视频的流化服务:



Application Layer: 2-113

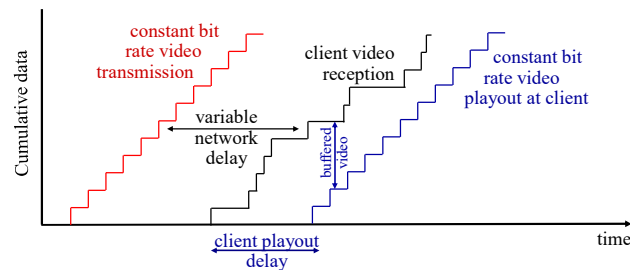
存储视频流化: 挑战

- **连续播放的限制:** 在客户端视频播放期间, 播放时间必须与视频原始时间一致-一旦播放就应该连续播放
 - ... 但是网络延迟是变化的 (存在抖动), 因此需要客户端的缓存来满足连续播放限制
- 其他的挑战:
 - 客户交互行为: 暂停, 前进, 倒退, 跳转
 - 视频数据包可能丢失、重新传输



Application Layer: 2-114

存储视频流化: 播放缓存



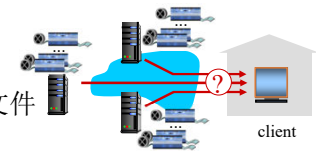
- **客户端缓存和播放延迟:** 补偿网络延迟和抖动

Application Layer: 2-115

多媒体流化: DASH

服务器:

- 视频被分割成多个视频块
- 每一块可以被编码成不同码率
- 每一块的每种码率的视频块被存为不同文件
- 文件被复制在各个的CDN节点上
- *manifest file*: 提供不同视频块的URL文件



客户端:

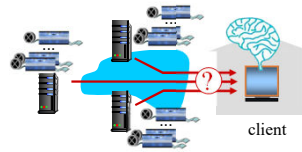
- 周期性估计服务器到客户端的带宽情况
- 查询货物清单, 在一个时刻请求一个块
 - 如果带宽足够, 选择最大码率的视频块
 - 会话中的不同时刻, 可以切换请求不同的编码块 (取决于当时的可用带宽), 从不同服务器那里

Application Layer: 2-116

2025中科大网络

多媒体流化: DASH

- “智能”客户端: 客户端自适应决定
 - 什么时候去请求块 (不至于缓存挨饿, 或者溢出)
 - 请求什么编码速率的视频块 (当带宽够用, 请求高质量的视频块)
- 哪里去请求块 (可以向离自己近的服务器发送URL, 或者向高可用带宽的服务器请求)



Streaming video = encoding + DASH + playout buffering

Application Layer: 2-117

2025中科大网络

Content distribution networks (CDNs)

挑战: 如何将内容 (从数百万个视频中选择) 流化服务器给数十万同时使用的用户?

- **选项1:** 单个的、大的超级服务中心
 - 单点故障
 - 网络拥塞
 - 服务器-客户端过长的 (可能拥塞) 路径

相当简单, 但是这个方法不可扩展

Application Layer: 2-118

2025中科大网络

Content distribution networks (CDNs)

挑战: 如何将内容 (从数百万个视频中选择) 流化服务器给数十万同时使用的用户?

- **选项2:** 通过CDN, 通过多个地理分布的站点存储视频副本, 提供流化服务
 - **enter deep:** 将CDN服务器深入到许多接入网
 - 更接近用户
 - Akamai: 在多于120个国家部署了240,000 服务器
 - **bring home:** 在一些关键位置 (离若干1stISP POP较近), 部署少数 (10个左右) 大型服务器簇
 - Limelight使用

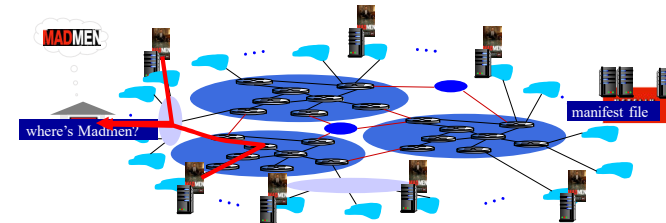


Application Layer: 2-119

2025中科大网络

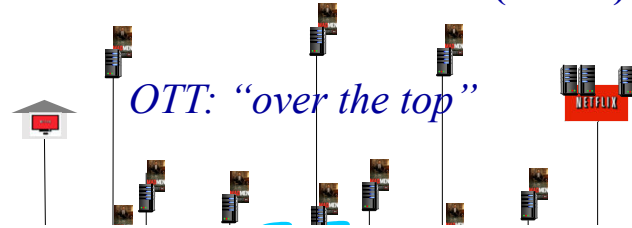
Content distribution networks (CDNs)

- CDN: 在CDN节点上存储内容的副本
- 订户请求内容, 服务器返回manifest文件
 - 使用manifest, 客户端以最高支持速率请求内容
 - 如果网络路径拥塞, 可以选择不同的速率或者副本



Application Layer: 2-120

Content distribution networks (CDNs)



互联网主机-主机之间的通信作为一种服务向用户提供

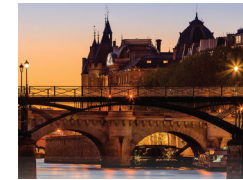
OTT 挑战: 在拥塞的互联网上从边缘复制内容

- 哪些内容被哪些CDN节点缓存?
- 从哪个CDN接地请求内容, 以什么速率

Application Layer: 2-121

应用层

- 应用层协议原理
- Web和HTTP
- E-mail, SMTP, IMAP
- DNS (The Domain Name System)
- P2P 应用
- 视频流化和CDN (content distribution networks)
- **UDP和TCP Socket编程**

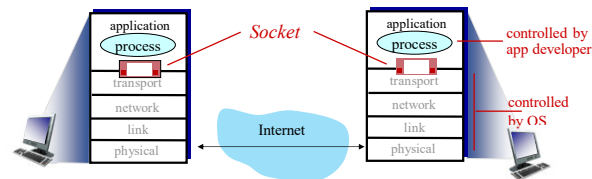


Application Layer: 2-122

Socket 编程

目标: 学习借助传输层提供的Socket API, 构建C/S模式应用

Socket: 分布式应用进程之间的门, 由传输层提供的进程到进程 (端到端) 服务接口



Application Layer: 2-123

Socket编程

2种类型的传输层Socket服务:

- **UDP:** 不可靠数据报
- **TCP:** 可靠字节流

应用例子:

1. 客户端用户从键盘输入一行字符, 发送给服务器
2. 服务器接收到该行字符, 转换成大写
3. 服务器发送转换之后的数据给客户端
4. 客户端接收转换之后的数据, 显示在显示器上

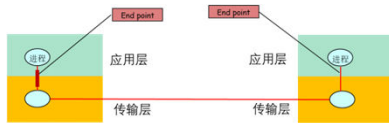
Application Layer: 2-124

数据结构 - sockaddr_in

表示IP地址和port捆绑关系的数据结构（标示进程间通信的一个端节点）

```
struct sockaddr_in {
    short sin_family; //AF_INET
    u_short sin_port; // port
    struct in_addr sin_addr;
    // IP address, unsigned long
    char sin_zero[8]; // align
};
```

结构体域	作用
short sin_family	地址类型
u_short sin_port	端口号
struct in_addr sin_addr	IP地址
char sin_zero[8];	对齐



Application Layer: 2-125

数据结构 - hostent

域名和IP地址的数据结构

```
struct hostent
{
    char *h_name; /*主机域名*/
    char **h_aliases; /*主机别名指针列表*/
    int h_addrtype; /*地址类型*/
    int h_length; /*地址长度*/
    char **h_addr_list; /*返回IP地址指针列表*/
    #define h_addr h_addr_list[0];
    /*第一个IP地址指针别名*/
}
```

202.38.64.1
202.38.64.3
202.38.64.8

作为调用DNS域名解析函数时的返回值
之后，应用程序可将IP地址拷贝到 sockaddr_in的IP地址部分

Application Layer: 2-126

UDP Socket编程

UDP: 在客户端和服务端之间无连接

- 在发送数据前没有握手，所建立的Socket只与本地IP、UDP端口号捆绑
- 发送方在调用UDP Socket发送每个数据报发送时，都要显式指定对方的端接点
- 接收方通过UDP Socket接收每个数据报中，须提取对方（发送方）的端接点：

用sockaddr_in结构变量存储

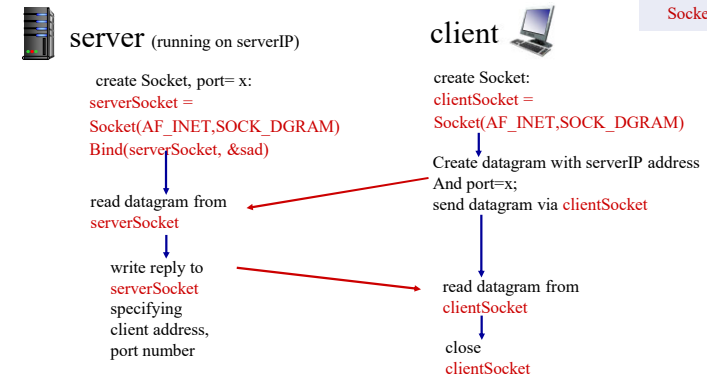
UDP: 发送的数据报可能丢失或者失序

应用视角:

- UDP在分布式进程间提供了不可靠的字节组（数据报）传输服务

Application Layer: 2-127

客户端/服务器Socket交互: UDP



Socket API

Application Layer: 2-128

样例: C客户端 (UDP)

2025中科大网络

```

/* client.c */
void main(int argc, char *argv[])
{
    struct sockaddr_in sad; /* structure to hold an IP address */
    int clientSocket; /* Socket descriptor */
    struct hostent *ptrh; /* pointer to a host table entry */

    char Sentence[128];
    char modifiedSentence[128];

    host = argv[1]; port = atoi(argv[2]);

    clientSocket = Socket(PF_INET, SOCK_DGRAM, 0);

    /* determine the server's address */
    memset((char *)&sad, 0, sizeof(sad)); /* clear sockaddr structure */
    sad.sin_family = AF_INET; /* set family to Internet */
    sad.sin_port = htons((u_short)port);
    ptrh = gethostbyname(host);
    /* Convert host name to IP address */
    memcpy(&sad.sin_addr, ptrh->h_addr, ptrh->h_length);

```

创建客户端 Socket, 和IP地址、
本地某个端口隐式捆绑
没有连接到服务器

```

struct sockaddr_in {
    short sin_family; /* AF_INET
    u_short sin_port; /* port
    struct in_addr sin_addr;
    // IP address, unsigned long
    char sin_zero[8]; // align
};

struct hostent {
    char *h_name; /* 主机域名 */
    char **h_aliases; /* 主机别名指针列表 */
    int h_addrtype; /* 地址类型 */
    int h_length; /* 地址长度 */
    char **h_addr_list; /* IP地址指针列表 */
    #define h_addr h_addr_list[0];
    /* 第一个IP地址指针别名 */
};

```

样例: C客户端 (UDP) (续)

2025中科大网络

```

Get
input stream from user ] gets(Sentence);

Send line
to server ] addr_len = sizeof(struct sockaddr);
n = sendto(clientSocket, Sentence, strlen(Sentence)+1,
           (struct sockaddr *) &sad, addr_len);

Read line
from server ] n = recvfrom(clientSocket, modifiedSentence, sizeof(modifiedSentence),
                        (struct sockaddr *) &sad, &addr_len);

printf("FROM SERVER: %s\n", modifiedSentence);

Close
connection ] close(clientSocket);
}

```

130

样例: C服务器(UDP)

2025中科大网络

```

/* server.c */
void main(int argc, char *argv[])
{
    struct sockaddr_in sad; /* structure to hold an IP address */
    struct sockaddr_in cad;
    int serverSocket; /* Socket descriptor */
    struct hostent *ptrh; /* pointer to a host table entry */

    char clientSentence[128];
    char capitalizedSentence[128];

    port = atoi(argv[1]);

    serverSocket = Socket(PF_INET, SOCK_DGRAM, 0);
    memset((char *)&sad, 0, sizeof(sad)); /* clear sockaddr structure */
    sad.sin_family = AF_INET; /* set family to Internet */
    sad.sin_addr.s_addr = INADDR_ANY; /* set the local IP address */
    sad.sin_port = htons((u_short)port); /* set the port number */
    bind(serverSocket, (struct sockaddr *)&sad, sizeof(sad));

```

Create welcoming Socket at port
&
Bind a local address

```

struct sockaddr_in {
    short sin_family; /* AF_INET
    u_short sin_port; /* port
    struct in_addr sin_addr;
    // IP address, unsigned long
    char sin_zero[8]; // align
};

```

131

样例: C服务器(UDP) (续)

2025中科大网络

```

while(1) {
    n = recvfrom(serverSocket, clientSentence, sizeof(clientSentence), 0,
                (struct sockaddr *) &cad, &addr_len);

    /* capitalize Sentence and store the result in capitalizedSentence */

    n = sendto(serverSocket, capitalizedSentence, strlen(capitalizedSentence)+1,
               (struct sockaddr *) &cad, &addr_len);

    }
}

```

Receive messages from clients

Write out the result to Socket

End of while loop,
loop back and wait for
another client connection

132

TCP Socket 编程

服务器等待来自用户的TCP连接建立请求

- 服务器进程必须首先运行
- 服务器必须**创建**welcome Socket (并**bind**)，准备**accept**来自客户端的连接建立请求

客户端主动联系服务器

- 创建TCP Socket，隐式**捆绑Bind**自己的某个闲置端口
- 用该Socket，客户端**建立起与服务器**的TCP连接，参数中指明服务器的IP地址和端口号

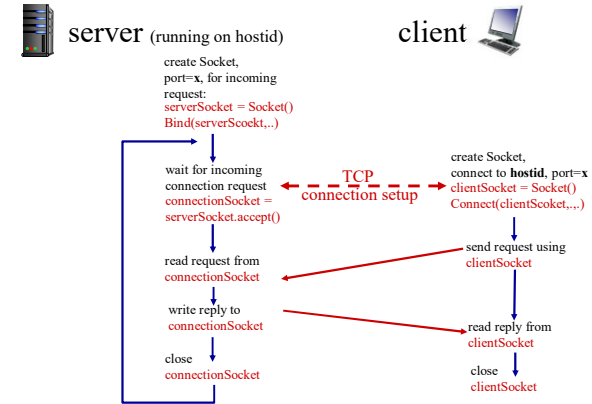
- 当服务器被客户端请求建立连接，**accept**解除阻塞，TCP弹出一个新创建的**connection Socket**，用于和特定的客户端进行通信
 - 允许服务器同时通过一个端口（但是不同的Socket）与多个客户端通信
 - 用客户端源IP和源端口，区分不同的客户端

应用视角

TCP在客户端和服务器之间提供可靠、保序的字节流(管道)传输服务

Application Layer: 2-133

客户端/服务器Socket交互: TCP



Application Layer: 2-134

C客户端(TCP)

```
/* client.c */
void main(int argc, char *argv[])
{
    struct sockaddr_in sad; /* structure to hold an IP address of server */
    int clientSocket; /* Socket descriptor */
    struct hostent *ptrh; /* pointer to a host table entry */

    char Sentence[128];
    char modifiedSentence[128];

    host = argv[1]; port = atoi(argv[2]);

    clientSocket = Socket(PF_INET, SOCK_STREAM, 0);
    隐式bind memset((char *)&sad, 0, sizeof(sad)); /* clear sockaddr structure */
    sad.sin_family = AF_INET; /* set family to Internet */
    sad.sin_port = htons((u_short)port);
    ptrh = gethostbyname(host);
    /* Convert host name to IP address */
    memcpy(&sad.sin_addr, ptrh->h_addr, ptrh->h_length);
    //将IP地址拷贝到sad.sin_addr
    connect(clientSocket, (struct sockaddr *)&sad, sizeof(sad));
}
```

Create client Socket,
connect to server

```
struct sockaddr_in {
    short sin_family; //AF_INET
    u_short sin_port; // port
    struct in_addr sin_addr;
    // IP address, unsigned long
    char sin_zero[8]; // align
};

struct hostent {
    char *h_name; /* 主机域名 */
    char **h_aliases; /* 主机别名指针列表 */
    int h_addrtype; /* 地址类型 */
    int h_length; /* 地址长度 */
    char **h_addr_list; /* IP地址指针列表 */
    #define h_addr h_addr_list[0]
    /* 第一个IP地址指针别名 */
}
```

Application Layer: 2-135

例子: C客户端(TCP) (续)

```
Get input stream from user } gets(Sentence);

Send line to server } n=write(clientSocket, Sentence, strlen(Sentence)+1);

Read line from server } n=read(clientSocket, modifiedSentence, sizeof(modifiedSentence));

printf("FROM SERVER: %s\n", modifiedSentence);

Close connection } close(clientSocket);
}
```

Application Layer: 2-136

例子: C服务器 (TCP)

2025中科大网络

```
/* server.c */
void main(int argc, char *argv[])
{
    struct sockaddr_in sad; /* structure to hold an IP address of server*/
    struct sockaddr_in cad; /*client */
    int welcomeSocket, connectionSocket; /* Socket descriptor */
    struct hostent *ptrh; /* pointer to a host table entry */

    char clientSentence[128];
    char capitalizedSentence[128];

    port = atoi(argv[1]);

    welcomeSocket = Socket(PF_INET, SOCK_STREAM, 0);
    memset((char *)&sad, 0, sizeof(sad)); /* clear sockaddr structure */
    sad.sin_family = AF_INET; /* set family to Internet */
    sad.sin_addr.s_addr = INADDR_ANY; /* set the local IP address */
    sad.sin_port = htons((u_short)port); /* set the port number */
    bind(welcomeSocket, (struct sockaddr *)&sad, sizeof(sad));
```

Create welcoming Socket at port
&
Bind a local address

```
struct sockaddr_in {
    short sin_family; //AF_INET
    u_short sin_port; // port
    struct in_addr sin_addr;
    // IP address, unsigned long
    char sin_zero[8]; // align
};
```

Application Layer: 2-137

例子: C服务器 (TCP) (续)

2025中科大网络

```
/* Specify the maximum number of clients that can be queued */
listen(welcomeSocket, 10)

while(1) {
    connectionSocket=accept(welcomeSocket, (struct sockaddr *)&cad, &alen);

    n=read(connectionSocket, clientSentence, sizeof(clientSentence));

    /* capitalize Sentence and store the result in capitalizedSentence */

    n=write(connectionSocket, capitalizedSentence, strlen(capitalizedSentence)+1);

    close(connectionSocket);
}
```

Wait, on welcoming Socket
for contact by a client

Write out the result to Socket

End of while loop,
loop back and wait for
another client connection

Application Layer: 2-138

第二章：总结

2025中科大网络

关于网络应用层的学习结束了!

- 应用架构
 - client-server
 - P2P
- 应用需要传输层提供服务的指标:
 - 可靠性、带宽和延迟
- 互联网传输层服务模型
 - 面向连接的可靠服务: TCP
 - 不可靠的数据报服务: UDP
- 流行应用的协议实例:
 - HTTP
 - SMTP, IMAP
 - DNS
 - P2P: BitTorrent
- 视频流化, CDNs
- Socket编程: TCP, UDP Sockets

Application Layer: 2-139

第二章：总结

2025中科大网络

最重要的是：了解了协议！

- 典型的请求/回复报文交换:
 - 客户端向服务器请求
 - 服务器以数据和状态码响应请求
- 报文格式:
 - 头部: 有关于数据的描述信息
 - 数据: 通信的信息 (载荷)

主要主题:

- 集中式 vs 分布式
- 无状态 vs. 有状态
- 可扩展性
- 可靠的vs. 不可靠报文传输
- 复杂性放置在网络边缘

Application Layer: 2-140

第二章幻灯片附件

Application Layer: 2-141

简单SMTP交互例子

```

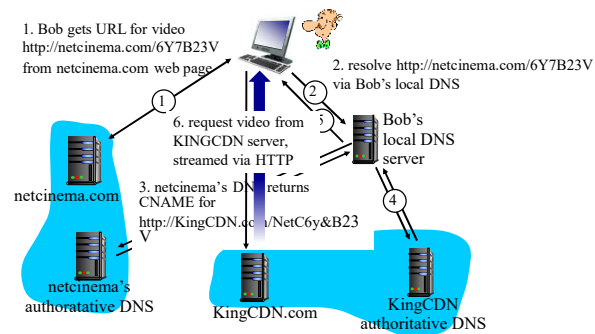
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
  
```

Application Layer: 2-142

CDN内容存取: 详细步骤

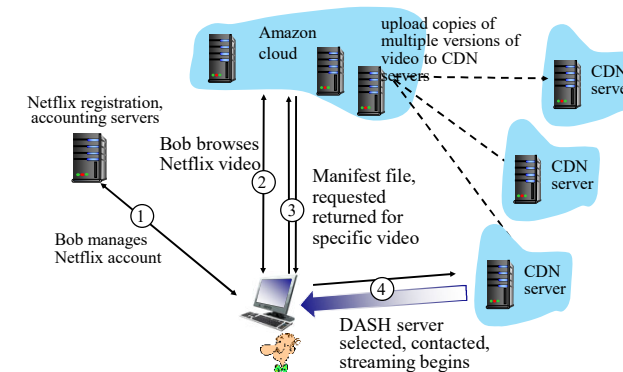
Bob (客户端) 请求视频 <http://netcinema.com/6Y7B23V>

- 视频被存储在 CDN 站点是: <http://KingCDN.com/NetC6y&B23V>



Application Layer: 2-143

案例学习: Netflix



Application Layer: 2-144